

DOC-09

VIXOR ENGINEERING BIBLE

كتاب الأمن السيبراني

الوثيقة الرسمية لمعايير وممارسات الأمن في فيكسور

جدول المحتويات

المصادقة (Authentication)

التفويض (Authorization)

إدارة الجلسات (Session Management)

رموز الويب (JSON (JWT

الأسرار (Secrets)

متغيرات البيئة (Environment Variables)

أمان واجهات البرمجة (API Security)

تقييد المعدل (Rate Limiting)

سياسة أمان المحتوى (CSP)

البرمجة عبر المواقع (XSS)

تزوير الطلبات (CSRF)

حقن (SQL (SQL Injection

سجلات التدقيق (Audit Logs)

التشفير (Encryption)

الاستجابة للحوادث (Incident Response)

المصادقة (Authentication)

تتبنى فيكسور نموذج مصادقة فريدًا يعتمد على **Telegram** كقناة أولية للمصادقة، مع الاحتفاظ بـ **البريد الإلكتروني** كآلية بديلة. النهج على حقيقة أن مستخدم فيكسور يصلون للتطبيق عبر **Telegram WebApp**، مما يمنحنا مصادقة موثوقة مسبقًا عبر **Telegram**. عند فتح التطبيق، يتم استلام بيانات **initData** المشفرة من Telegram والتي تحتوي على معرف المستخدم والتوثيق. تُمرّر هذه البيانات إلى دالة **telegramSignIn** الموجودة في **domains/user/auth.functions.ts** للتحقق الرقمي عبر **TELEGRAM_BOT_TOKEN** وإنشاء المستخدم إذا كان جديدًا.

✦ ميزة Telegram-First

لا يحتاج المستخدم إلى تسجيل حساب منفصل أو تذكر كلمة مرور. المصادقة تتم تلقائيًا عبر حساب Telegram الموجود بالفعل، مما يقلل التسجيل إلى صفر ويضمن هوية موثوقة عبر البنية الأمنية لـ Telegram.

تدفق المصادقة الكامل

يبدأ التدفق عند فتح **Login Widget** في واجهة المستخدم، والذي يستدعي **telegramSignIn** ويمرر تجزئة **hash** كملف **TELEGRAM_BOT_TOKEN** مع HMAC-SHA256 للتحقق من صحة التجزئة باستخدام خوارزمية Supabase Auth عبر **JWT token**. في مس المحمية، يتحقق **routes/_authenticated/route.tsx** — المُعد بـ **ssr: false** — من صلاحية الـ **beforeLoad** قبل عرض أي محتوى.

- **WebApp Auto-Signin**: المصادقة التلقائية عند فتح التطبيق من Telegram عبر **initData**
- **Login Widget**: واجهة تسجيل الدخول اليدوية للاستخدام خارج Telegram
- **Email Fallback**: تسجيل الدخول بالبريد الإلكتروني وكلمة المرور كآلية احتياطية
- **telegramSignIn**: الدالة المركزية للتحقق من تجزئة Telegram وإنشاء المستخدم

التفويض (Authorization)

بعد نجاح المصادقة، ينتقل النظام إلى مرحلة **التفويض** التي تحدد ما يسمح للمستخدم بالوصول إليه. يعتمد فيكسور على ط التفويض تبدأ من **Row Level Security (RLS)** في Supabase وصولاً إلى حراس المصادقة (Auth Guards) المُطبّقين على API. تضمن سياسات RLS أن المستخدم لا يستطيع الوصول إلا إلى بياناته الخاصة، بينما تتحقق حراس المصادقة من صلاحيات كل طلب.

Row Level Security (RLS)

تُفعّل سياسات RLS على جميع الجداول الحساسة في قاعدة البيانات. تعمل هذه السياسات كجدران حماية على مستوى الصف حيث يتم فحص كل استعلام ضد قواعد التفويض المحددة في Supabase. يستخدم الوسيط **auth-middleware.ts** دالتين مختلفتين لإنشاء عملاء Supabase: عميل **Validation** للتحقق من هوية المستخدم، و عميل **shared/supabase/requireSupabaseAuth** لاستعلامات ضمن سياق أمان المستخدم. دالة **requireSupabaseAuth** تستخرج رمز Bearer من رأس الطلب وتُمرره للعميلين.

✦ حراس المصادقة على مستوى API

كل نقطة نهاية Nitro (14 نقطة) مُزوّدة بحارس مصادقة يحدد مستوى الوصول المطلوب: Admin/Cron للمهام الداخلية، Bearer Auth للمستخدمين المُصدّق عليهم، Webhook Secret لعمليات التحقق من المصدر، Health Token لفحوصات الحالة، أو Public للنقاط المفتوحة.

يتمتع المديرون بصلاحيات واسعة تشمل الوصول إلى جميع سجلات المستخدمين وإدارة تكوين النظام ومراجعة سجلات الت الوصول الإداري وجود `ADMIN_KEY` في رأس الطلب، وهو مفتاح سري مشفر يُخزّن حصريًا في متغيرات بيئة الخادم. يتم الت المفتاح في طبقة الأمان `_security.ts` قبل السماح بتنفيذ أي عملية إدارية.

مستوى الوصول	آلية التحقق	نطاق الصلاحيات
Admin/Cron	ADMIN_KEY أو CRON_SECRET	وصول كامل إلى جميع البيانات والعمليات
Bearer Auth	JWT صالح من Supabase	بيانات المستخدم الخاص فقط
Webhook	TELEGRAM_WEBHOOK_SECRET	معالجة أحداث Telegram الواردة
Health	Health token مُشترَك	فحص حالة النظام فقط
Public	بدون تحقق	بيانات عامة محدودة

إدارة الجلسات (Session Management)

تتبنى فيكسور نموذجًا عديم الحالة (Stateless) لإدارة الجلسات يعتمد بالكامل على `JWT (JSON Web Tokens)` الصادرة عن Auth. لا يوجد تخزين مركزي للجلسات على الخادم، بل يتم ترميز جميع معلومات الجلسة داخل الرمز المميز نفسه. هذا النهج التحتية ويتوافق مع طبيعة التطبيق كـ WebApp داخل Telegram حيث تكون الجلسات قصيرة ومتكررة.

دورة حياة الجلسة

تبدأ الجلسة عند نجاح المصادقة وتلقي العميل رمز `access_token` و `refresh_token`. يُخزّن رمز الوصول في `cookie` آمن ويُرسل مع كل طلب API عبر رأس `Authorization: Bearer`. عند انتهاء صلاحية رمز الوصول (عادةً بعد ساعة واحدة) العميل رمز التحديث للحصول على رمز وصول جديد تلقائيًا دون تدخل المستخدم. تتم إدارة هذه العملية عبر آلية التحديث `supabase-js` التي تعمل بشفافية في الخلفية.

رمز التحديث (Refresh Token)

رمز طويل العمر يُستخدم فقط لاستبدال رمز الوصول المنتهية بشكل آمن ويُتبادل مع رمز وصول جديد فقط عبر نقطة نها

رمز الوصول (Access Token)

رمز JWT قصير العمر (ساعة واحدة) يُستخدم للمصادقة على كل طلب. يحتوي على معرف المستخدم والصلاحيات ووقت الانتهاء. لا يُخزّن أبدًا في LocalStorage.

⚠ سياسة الأمان للجلسات

يُمنع تمامًا تخزين الرموز المميزة في `localStorage` أو `sessionStorage`. جميع الرموز تُخزّن حصريًا في `HttpOnly Cookies` مع `Secure` و `SameSite=Strict` لتقليل مخاطر سرقة الجلسات عبر هجمات XSS و CSRF.

مخزن Supabase الآمن

يتم الوصول إلى عميل Supabase من جانب المتصفح عبر دالة `getSupabaseOrNull` أو `getSupabaseOrThrow` (Singleton) وإعادة استخدام `Lazy Initialization` (التهيئة الكسولة). يضمن هذا النمط إنشاء عميل Supabase لمرة واحدة فقط (Singleton) وإعادة استخدام أنحاء التطبيق. هذا يمنع تسرب الاتصالات ويضمن اتساق حالة المصادقة عبر جميع المكونات.

رموز الويب (JWT) JSON

JSON Web Token

هو المعيار الأساسي لمصادقة الطلبات في فيكسور. يتكون كل رمز JWT من ثلاثة أجزاء مفصولة بنقطة. يحتوي على نوع الرمز وخوارزمية التوقيع، و**Payload** يحتوي على المطالبات (Claims) مثل معرف المستخدم والصلاحيات والانتها، و**Signature** وهو التوقيع الرقمي الذي يضمن سلامة الرمز. تُصدر جميع رموز JWT عبر **Supabase Auth** باستخدام السري الخاص بقاعدة البيانات.

بنية الرمز المميز

```
{ "alg": "HS256", "typ": "JWT" }
{ "sub": "uuid", "role": "authenticated",
  "iat": 1700000000, "exp": 1700003600,
  "app_metadata": { "provider": "telegram" } }
e: HMAC-SHA256(base64(header) + "." + base64(payload), SECRET)
```

عند استلام طلب API، يستخرج وسيط **requireSupabaseAuth** رمز Bearer من رأس الطلب ويُمرره إلى Supabase Supabase بالتحقق من التوقيع الرقمي وفحص تاريخ الانتهاء وتأكيد أن المستخدم لا يزال نشطًا. في حالة نجاح التحقق Supabase مُهيأ بسياق أمان المستخدم (RLS Context) لتنفيذ الاستعلامات اللاحقة ضمن حدود صلاحياته.

✦ عملية التحقق من الرمز

1. استخراج الرمز من رأس <Authorization: Bearer <token
2. التحقق من صحة التوقيع باستخدام SUPABASE_ANON_KEY
3. فحص تاريخ الانتهاء (exp) والمطالبة الخاصة بالمنشأ (iss)
4. إنشاء عميل Supabase مع سياق RLS الخاص بالمستخدم

انتهاء الصلاحية والتحديث

يُحدّد عمر رمز الوصول بـ **3600 ثانية** (ساعة واحدة) كتوازن بين الأمان وأداء المستخدم. عند انتهاء الصلاحية، يُرسل العميل رمز نقطة نهاية **/auth/token** في Supabase للحصول على رمز وصول جديد. تتم هذه العملية تلقائيًا وبشكل شفاف دون الحاجة الصفحة. في حالة انتهاء صلاحية رمز التحديث أيضًا (بعد 30 يومًا افتراضيًا)، يُطلب من المستخدم إعادة المصادقة.

الأسرار (Secrets)

تُعد إدارة الأسرار من أعمدة الأمان الأساسية في فيكسور. يُعرّف السر بأنه أي قطعة بيانات حساسة لا يجب أن تُكشف أبدًا — API، رموز الوصول، كلمات المرور، وبيانات الاعتماد المشفرة. تتبنى فيكسور مبدأ **أقل الامتيازات (Principle of Least Privilege)** يحصل كل مكون على الأسرار الضرورية فقط لعمله، ولا شيء أكثر.

تصنيف الأسرار

السياق	الاستخدام	السّر
الخادم والعميل	عنوان نقطة نهاية Supabase	SUPABASE_URL
الخادم والعميل	مفتاح Supabase العام مع RLS	SUPABASE_ANON_KEY
الخادم فقط	مفتاح Supabase الإداري (يتجاوز RLS)	SUPABASE_SERVICE_ROLE_KEY
الخادم فقط	مصادقة وتحقق بيانات Telegram	TELEGRAM_BOT_TOKEN
الخادم فقط	التحقق من مصدر Webhooks	TELEGRAM_WEBHOOK_SECRET
الخادم فقط	مصادقة مهام Vercel Cron	CRON_SECRET
الخادم فقط	الوصول الإداري الكامل	ADMIN_KEY
الخادم فقط	اتصال Redis للتخزين المؤقت	REDIS_URL

⚠ قاعدة ذهبية: لا تُرسل الأسرار للعميل أبدًا

أي سري لا يحمل بادئة `_VITE` لا يمكن الوصول إليه من جانب العميل. يفرض Vite هذا التقيد بشكل مدمج. السّر الوحيد المسموح للعميل `_VITE` * المتفق عليه والمُراجع آمنًا. جميع المفاتيح الحساسة مثل `SUPABASE_SERVICE_ROLE_KEY` و `TELEGRAM_BOT_TOKEN` و `ADMIN_KEY` تبقى حصرًا على الخادم.

مفاتيح مزودي LLM

تحتوي فيكسور على مفاتيح API لمزودي نماذج اللغة الكبيرة (LLM) المختلفين. تُخزّن هذه المفاتيح في متغيرات بيئة الخادم ولا إليها من العميل أبدًا. يتم تمريرها حصرًا عبر نقاط نهاية API المحمية التي تتحقق من هوية المستخدم قبل إعادة توجيه الطلبات لـ LLM. هذا يمنع استغلال مفاتيح API باهظة التكلفة من قبل جهات غير مصرح لها.

06

متغيرات البيئة (Environment Variables)

تعتمد فيكسور على نظام صارم لتصنيف متغيرات البيئة يضمن فصلًا واضحًا بين ما يمكن للعميل الوصول إليه وما يبقى حصرًا يتكامل هذا النظام مع آلية Vite المدمجة حيث يتم حقن المتغيرات التي تحمل بادئة `VITE_` في حزمة العميل (Client Bundle) البناء، بينما تظل جميع المتغيرات الأخرى محجوبة تمامًا عن جانب المتصفح.

قاعدة البادئة VITE_

تُعد بادئة `VITE_` البوابة الوحيدة لنقل البيانات من بيئة الخادم إلى العميل. عند بناء التطبيق، يستبدل Vite تلقائيًا `import.meta.env.VITE_*` بقيمتها الفعلية وقت البناء. هذا يعني أن أي متغير لا يحمل هذه البادئة سيكون `undefined` العميل حتى لو كان موجودًا في بيئة الخادم. يوفر هذا حماية مدمجة وقوية دون الحاجة لأي تكوين إضافي.

✦ سياق الخادم مقابل العميل

على الخادم: الوصول عبر `process.env.SECRET_KEY` (كامل الوصول).
على العميل: الوصول عبر `import.meta.env.VITE_PUBLIC_KEY` (مقيّد بالبادئة فقط).
الوسم `server.ts`: يمنع بشكل صارم تضمين كود الخادم في حزمة المتصفح.

فرض تنفيذ الخادم فقط

يستخدم فيكسور اصطلاح تسمية `server.ts`. كلاحقة لجميع الملفات التي تحتوي على كود يجب أن يُنقذ حصرًا على الخادم. (محرك الخادم) على هذه اللاحقة ويمنع تضمين هذه الملفات في حزمة العميل أثناء البناء. هذا يضمن أن الأسرار والمنطق الحس

تسريبه إلى المتصفح عن طريق الخطأ، حتى إذا تم استيراد الملف غير مباشر من مكون عميل.

- **VITE_البادئة:** العلامة الوحيدة المسموح بها لكشف متغير للعميل
- **process.env:** متاح حصريًا في سياق الخادم (Nitro endpoints, .server.ts)
- **server.ts:** ضمان بنائي (Build-time) لعدم تسرب كود الخادم
- **client.server.ts:** عميل Supabase الإداري يقتصر استخدامه على الخادم فقط

07

أمان واجهات البرمجة (API Security)

تمتلك فيكسور 14 نقطة نهاية Nitro محمية بطبقة أمان مركزية مُعرّفة في `_security.ts`. هذه الطبقة تعمل كجدار حماية من هوية المصدر وصلاحياته قبل السماح بتنفيذ أي منطق تجاري. تم تصميم هذه البنية وفقًا لمبدأ **Defense in Depth** (الطبقات) حيث يوجد أكثر من حاجز أمني بين المهاجم والبيانات الحساسة.

طبقة الأمان المركزية (`security.ts`)

ملف `_security.ts` يُعرّف أربعة مكونات رئيسية: **CORS** يقيّد origins المسموح بها إلى `vixor.app` و `vercel.app`، **Rate Limiting** عبر دالة `withRateLimit()` التي تُغلّف نقاط النهاية بمحددات معدل الطلبات، **Authentication** استراتيجيات مصادقة مختلفة (Bearer JWT, Admin Key, Vercel Cron, Cron Secret, Webhook Secret)، و **Authorization** الذي يُحوّل الأخطاء غير المعالجة إلى استجابات موحدة وآمنة.

Auth Strategies

خمس استراتيجيات: Bearer JWT للمستخدمين، Admin Key للمديرين، Cron Secret للمهام المجدولة، Telegram و Health Token لفحوصات الحالة. كل استراتيجية منطق تحقق خاص.

CORS Policy

تقييد صارم لمصادر الطلبات. يُسمح فقط بـ `vixor.app` و `*.vercel.app` أي طلب من مصدر خارجي يُرفض فورًا برمز 403. يمنع هذا هجمات Cross-Origin المحتملة.

توزيع حراس المصادقة على النقاط

نوع الحارس	العدد	أمثلة على الاستخدام
Admin/Cron	متعدد	إدارة المستخدمين، مهام التنظيف، التقارير الداخلية
Bearer Auth	الأغلبية	جميع عمليات المستخدم: التداول، المحفظة، الإعدادات
Webhook Secret	محدود	استقبال أحداث Telegram وعمليات الدفع عبر Stars
Health Token	واحد	نقطة فحص الحالة الصحية للنظام
Public	محدود	بيانات السوق العامة، معلومات الأسعار

08

تقييد المعدل (Rate Limiting)

تُعد الحماية من إساءة الاستخدام (Abuse Prevention) ركناً أساسيًا في بنية فيكسور الأمنية. توفر المنظومة ثلاث آليات مختلفة للطلبات (Rate Limiting) تُستخدم حسب السياق والاحتياج: **SlidingWindowLimiter** للتحكم المحلي، **redisRateLimiter** لمثلثات متعددة، و **withRateLimit()** كغلاف مريح يُطبّق على نقاط النهاية. تمنع هذه الآليات هجمات **Brute Force** و **DDoS**.

محدد النافذة المنزلقة (`SlidingWindowLimiter`)

يُنقَد **SlidingWindowLimiter** خوارزمية النافذة المنزلقة محليًا في ذاكرة العملية. يتتبع عدد الطلبات خلال إطار زمني متحرك (م) في 60 ثانية). عند تجاوز الحد المسموح، يُرفض الطلب برمز **429 Too Many Requests**. يُفصّل هذا المحدد في السيناريو التالي. تتطلب استجابة فورية دون اعتماد على شبكة خارجية، مثل حماية نقاط النهاية من الاستخدام المفرط من عميل واحد.

محدد Redis الموزع (RedisRateLimiter)

البيئات التي تتطلب مزامنة عبر عدة مثيلات خادم، يُستخدم **RedisRateLimiter** الذي يعتمد على **REDIS_URL** لتخزين عدد في ذاكرة تخزين مؤقت موزعة. يضمن هذا أن الحد الأقصى يُحترم حتى إذا تم توزيع الطلبات على خوادم مختلفة. يتم استخدام **Window Log** في Redis لتتبع الطوابق الزمنية لكل طلب وتحديد ما إذا كان الحد قد تم تجاوزه.

✦ دالة withRateLimit() المريحة

`withRateLimit(handler, options)` هي دالة تغليف عالية المستوى تُطبّق على أي معالج نقطة نهاية Nitro. تقوم تلقائيًا باختبار المناسب (محلي أو Redis) بناءً على التكوين، وتُرجع استجابة 429 مع رأس `Retry-After` عند التجاوز. تبسط هذه الدالة تطبيق التقييد boilerplate ل إضافي.

قاطع الدائرة (CircuitBreaker)

إلى جانب تقييد المعدل، تُستخدم نمطية **CircuitBreaker** لحماية النظام من التسلسلات الفاشلة. عندما يفشل عدد معين المتتالية إلى خدمة خارجية (مثل مزود LLM أو Binance API)، يفتح القاطع الدائرة ويمنع إرسال طلبات إضافية لفترة محددة (Period). بعد انتهاء هذه الفترة، يسمح القاطع بطلب تجريبي واحد — إذا نجح، يُغلق القاطع وتعود العمليات الطبيعية. هذا يمنع المتتالية (Cascade Failures) ويحافظ على استقرار النظام.

سياسة أمان المحتوى (CSP)

تُعرّف **سياسة أمان المحتوى (Content Security Policy)** في `vite.config.ts` كطبقة حماية أساسية ضد هجمات تعمل CSP كقائمة بيضاء (Whitelist) تُحدد بدقة المصادر المسموح لها بتحميل الموارد في تطبيق فيكسور. أي محتوى من مصدر غير مسموح به في القائمة البيضاء يُحظر تلقائيًا بواسطة المتصفح، مما يمنع تنفيذ البرمجيات الخبيثة حتى في حالة وجود ثغرة XSS.

القائمة البيضاء المعتمدة

المجال المسموح	الغرض	الأنواع المسموحة
tradingview.com	رسوم بيانية مدمجة	frames, scripts, styles
telegram.org	WebApp SDK	frames, scripts
supabase.co	مصادقة وبيانات	connect, frames
binance.com	بيانات التداول	connect, images
dexscreener.com	بيانات DEX	connect, images
birdeye.so	تحليلات On-chain	connect
helius.xyz	Solana RPC	connect
twitter.com	تكامل وسائل التواصل	frames, images
lunarcrush.com	بيانات المشاعر	connect
coingecko.com	أسعار العملات	connect, images

تُعد توجيهات frame-ancestors ضرورية لتشغيل فيكسور ك WebApp داخل Telegram. بدونها سيمنع المتصفح تحميل التطبيق Telegram. يتم تحديد telegram.org كمصدر أبا (parent origin) مسموح لضمان التشغيل الصحيح داخل بيئة Telegram WebApp.

توجيهات CSP الأساسية

- **'default-src' self**: السماح بالموارد من نفس المصدر فقط افتراضياً
- **script-src**: السماح بـ inline scripts محددة و scripts من المصادر الموثوقة
- **connect-src**: تحديد نقاط النهاية المسموح باتصال fetch/WebSocket إليها
- **frame-src**: التحكم في الإطارات المضمنة (TradingView, Telegram WebApp)
- **img-src**: السماح بالصور من المصادر المعتمدة مع دعم data: URIs
- **style-src**: السماح بأنماط inline و external من المصادر الموثوقة

10

البرمجة عبر المواقع (XSS)

Cross-Site Scripting (XSS) هو أحد أخطر التهديدات الأمنية لتطبيقات الويب، حيث يقوم المهاجم بحقن برمجيات صفحات يراها مستخدمون آخرون. تتبنى فيكسور استراتيجية **Defense in Depth** متعددة الطبقات للوقاية من XSS، تبدأ من نفسه وصولاً إلى سياسات CSP الصارمة. تضمن هذه الاستراتيجية أنه حتى في حالة وجود ثغرة في طبقة واحدة، تمنع الطبقات استغلالها.

الحماية المدمجة في React

يوفر إطار **React** تلقائياً حماية قوية ضد XSS عبر آلية **Auto-Escaping**. عند استخدام JSX لعرض المحتوى، يُهرب تلقائياً (e) الأحرف الخاصة مثل `<` و `>` و `"` و `&` قبل إدراجها في DOM. هذا يعني أن `<script>alert('xss')</script>` سيعادى بدلاً من تنفيذه ككود. تنطبق هذه الحماية افتراضياً على جميع مكونات React في فيكسور.

✖ نقاط الضعف المحتملة

الاستخدام المباشر لـ `dangerouslySetInnerHTML` أو `document.write()` أو `innerHTML` يلغي حماية **Auto-Escaping**. يُمكن هذه الوظائف إلا بعد مراجعة أمنية صارمة وتطبيق **DOMPurify** على المحتوى المدخل. أي استخدام يجب أن يكون مُعلّقاً بتعليق أمني يوضح السبب.

طبقات الحماية الإضافية

- 1 **CSP Headers**: تمنع تنفيذ أي script من مصدر غير مصرح به، حتى لو تم حقنه بنجاح في DOM
- 2 **HttpOnly Cookies**: تمنع JavaScript من الوصول إلى رموز الجلسات، مما يُبطل هجمات XSS السرقة
- 3 **Input Validation**: التحقق من صحة جميع المدخلات على الخادم قبل المعالجة أو التخزين
- 4 **Output Encoding**: ترميز مخصص للمحتوى المعروض في سياقات مختلفة (HTML, URL, CSS, JS)
- 5 **Content Sanitization**: عند الحاجة لعرض HTML (مثل وصف التوكن)، يُنظف عبر مكتبة معتمدة

التعامل مع المحتوى الخارجي

قد يحتاج التطبيق لعرض محتوى من مصادر خارجية مثل أوصاف العملات الرقمية من CoinGecko أو تغريدات من Twitter. الحالات، يُطبّق بروتوكول تنظيف صارم يتضمن: إزالة جميع العناصر `<script>` و `<iframe>` و `<object>`، إزالة معال مثل `onclick` و `onerror`، تقييد السمات المسموح بها إلى قائمة بيضاء صارمة، لجميع الروابط الخارجية `rel="noopener noreferrer">`

تزوير الطلبات (CSRF)

Cross-Site Request Forgery (CSRF) هو هجوم يُجبر المتصفح على إرسال طلبات غير مصرح تطبيق موثوق أثناء تصفح المستخدم لموقع ضار. تستفيد هذه الهجمات من حقيقة أن المتصفح تلقائيًا ملفات تعريف الارتباط (Cookies) مع كل طلب إلى النطاق المستهدف. تتبنى استراتيجية شاملة للوقاية من CSRF تعتمد على آليات متعددة مترابطة.

SameSite Cookies كخط دفاع أول

تُعد سمة **SameSite=Strict** على ملفات تعريف الارتباط أقوى حماية ضد CSRF. عند تفعيل هذه السمة، يُرسل المتصفح ملفات تعريف الارتباط فقط مع الطلبات التي تنشأ من نفس النطاق (Requests). أي طلب يأتي من موقع خارجي (Cross-Site) لن يحتوي على ملفات تعريف الارتباط. يُبطل هجوم CSRF تلقائيًا. تستخدم فيكسور هذا الإعداد على جميع ملفات تعريف الارتباط التي فيها رموز المصادقة.

Bearer Token Authentication

بدلاً من الاعتماد فقط على Cookies، تستخدم فيكسور رأس Authorization: Bearer للمصادقة على API. لا يمكن للمتصفح إرسال هذا الرأس تلقائيًا في طلبات cross-site، مما يُبطل هجوم CSRF.

SameSite=Strict

أقوى مستوى حماية. يمنع إرسال Cookies مع أي طلب cross-site، بما في ذلك الروابط العادية. يضمن أن الطلبات المُصدّقة عليها تأتي فقط من داخل التطبيق.

CORS كطبقة حماية إضافية

تعمل سياسة **CORS** كحاجز إضافي ضد CSRF من خلال التحقق من رأس **Origin** في كل طلب. تُقبل فقط من **vixor.app** و ***.vercel.app**، بينما يُرفض أي طلب من مصدر آخر. حتى لو تمكنت من تشغيل طلب من موقع ضار، فلن يتمكن من قراءة الاستجابة بسبب قيود CORS. تُعرّف هذه القيود في **_security.ts** وتُطبّق على جميع نقاط النهاية.

+ نموذج Telegram WebApp المقاوم لـ CSRF

بما أن فيكسور يعمل كـ Telegram WebApp، فإن جميع التفاعلات تتم داخل بيئة Telegram المُتحمّكة. لا يمكن لموقع خارجي محاكاة بيئة WebApp أو إرسال طلبات حاملة لـ **initData** صالح. يوفر هذا حماية طبيعية إضافية تتجاوز الآليات التقنية التقليدية.

حقن (SQL Injection)

SQL Injection هو هجوم يحاول المهاجم من خلاله حقن أكواد SQL خبيثة في استعلامات البيانات للوصول إلى بيانات غير مصرح بها أو تعديلها أو حذفها. تُعد فيكسور محصّنة ضد هذا النوع من الهجمات بفضل استخدام **Supabase Client** الذي يُنفّذ **Parameterized Queries** افتراضيًا، بالإضافة إلى سياسات **Row Level Security** التي تُقيّد الوصول على مستوى قاعدة البيانات نفسها.

الاستعلامات المُعلّمة (Parameterized Queries)

يستخدم **supabase-js** الاستعلامات المُعلّمة حصريًا لجميع عمليات قاعدة البيانات. في هذه الحالة، تُفصل معلمات الاستعلام عن هيكل الاستعلام نفسه، مما يمنع المحرك من تفسير أي مدخل كأمر.

سبيل المثال، عند استخدام `.eq('id', userInput)`، تُعامل قيمة `userInput` كقيمة بيانات كجزء من هيكل SQL. هذا يُلغي تمامًا إمكانية حقن SQL عبر المدخلات.

```
supabase-js يُستخدم parameterized queries داخليًا
data } = await supabase
  .from('agents')
  .select('*')
  .eq('id', userId)
  .order('created_at', { ascending: false });

في كود فيكسور raw SQL محطور تمامًا - لا
result = await db.raw(`SELECT * FROM agents WHERE id = ${userInput}`)
```

Row Level Security كحاجز أخير

حتى في السيناريو النظري الذي يتمكن فيه المهاجم من حقن SQL، توفر سياسات **RLS** طبقة إضافية على مستوى قاعدة البيانات. تتحقق RLS من هوية المستخدم الحالي (عبر `userId()` في Supabase) وتقيّد النتائج بناءً على ملكيته. هذا يعني أنه حتى استعلام SQL محقون الوصول إلى بيانات المستخدم المُصدّق عليه، وليس بيانات المستخدمين الآخرين. تُفعّل RLS الجداول الحساسة في قاعدة البيانات.

⚠ قاعدة صارمة: لا يُسمح بـ Raw SQL

يُمنع تمامًا استخدام استعلامات SQL الخام (Raw SQL) في كود فيكسور. جميع العمليات يجب أن تتم عبر `supabase-js Client` فقط. هذا يضمن أن جميع الاستعلامات مُعلّمة ومحمية. أي استثناء يتطلب مراجع أمنية رسمية وموافقة من فريق الأمان.

13

سجلات التدقيق (Audit Logs)

تتبنى فيكسور نظامًا شاملاً لسجلات التدقيق (Audit Logging) يُسجّل جميع الأنشطة المهمة لغرض المراجعة الأمنية والتحقيق في الحوادث ومراقبة الامتثال. يعتمد هذا النظام على **agent_audit_log** و **domain_events** و **Sentry** للمراقبة الخارجية عبر Sentry لالتقاط الأخطاء في الوقت الحقيقي و **Mixpanel** لتحليل سلوك المستخدمين.

جدول agent_audit_log

يُخزّن **agent_audit_log** جميع العمليات المتعلقة بالوكلاء الذكيين (AI Agents) في فيكسور. كل إجراء مع طابعه الزمني ومعرّف المستخدم المنقذ ونوع العملية والبيانات المتأثرة. هذا الجدول لمراجعة سلوك الوكلاء، وتحليل الأداء، والتحقيق في أي سلوك غير متوقع. العمليات المُسجّلة: إنشاء الوكيل، تعديل الإعدادات، تنفيذ الصفقات، والتغييرات في استراتيجيات التداول.

جدول domain_events

يُمثّل **domain_events** سجل الأحداث المجالية (Domain Events) الذي يُوثّق جميع التغييرات في حالة النظام وفقًا لنمط **Event Sourcing**. كل حدث يحتوي على: نوع الحدث، البيانات، طابع زمني، ومعرّف التجميع (Aggregate ID). يوفر هذا السجل تاريخًا كاملاً وقابلًا للتدقيق للتغيرات في النظام، مما يُسهّل إعادة بناء الحالة في أي نقطة زمنية والتحقيق في الحوادث.

Mixpanel - تحليل السلوك

يُتبع Mixpanel تفاعلات المستخدمين لتحليل أنماط الاستخدام واكتشاف الأنشطة المشبوهة. يُسجل كل حدث مهم (تسجيل دخول، صفقة، تعديلات إعدادات) مع بيانات سياقية تُسهّل التحقيق الأمني والتحليل الإحصائي.

Sentry - مراقبة الأخطاء

يلتقط Sentry جميع الأخطاء غير المُعالَجة في بيئة الإنتاج ويُرسل تنبيهات فورية لفريق التطوير. يتضمن كل حدث تتبع المكس (Stack Trace)، سياق المستخدم، ومعلومات الجهاز. يُستخدم DSN خاص بفيكسور لعزل الأخطاء عن المشاريع الأخرى.

+ توحيد معالجة الأخطاء

ملف `server.ts` يُطبّق توحيدًا للأخطاء الكارثية (Catastrophic Error Normalization) حيث يُحوّل الأخطاء غير المُعالَجة إلى استجابات آمنة. في بيئة التطوير: يُعرض Stack Trace كامل للمطورين. بيئة الإنتاج: يُرجع رسالة عامة آمنة مع تسجيل التفاصيل في Sentry لحماية المعلومات الحساسة.

14

التشفير (Encryption)

تُعد طبقة التشفير خط الدفاع الأخير في بنية فيكسور الأمنية، حيث تضمن أن البيانات المخزنة محمية حتى في حالة اختراق طبقات الدفاع السابقة. يتم تطبيق التشفير على مستويين: **بيانات الاعتماد (Credential Encryption)** على مستوى التطبيق، و **TLS** على مستوى النقل. فيكسور مكتبات `noble/ed25519@` و `tweetnacl` لتوفير تشفير عالي الأداء ومقاوم للتحليل.

تشفير بيانات الاعتماد (Credential Encryption)

تُخزّن بيانات الاعتماد الحساسة (مثل مفاتيح API الخاصة بالبورصات ورموز الوصول) في شكل **مُشفّر**، وليس كنص عادي. يوفر مجلد `shared/crypto/` دالتين `encryptCredential` لتشفير البيانات قبل التخزين، و `decryptCredential` لفك التشفير عند الحاجة. تعتمد هاتان الدالتان على خوارزميات **Ed25519** (و `Networking and Cryptography Library`) التي تُوفّر سرية وموثوقية عالية.

تشفير بيانات الاعتماد قبل الـ

```
encryptCredential, decryptCredential } from '@shared/crypto';
```

عند حفظ بيانات اعتماد بورصة

```
encrypted = encryptCredential(apiKey, secret);
```

```
database.from('exchange_credentials').insert({
```

```
id: userId,
```

```
exchange: 'binance',
```

```
encrypted_api_key: encrypted.key,
```

```
encrypted_api_secret: encrypted.secret,
```

عند استخدام بيانات الاعتماد

```
{ key, secret } = decryptCredential(cred);
```

```
client = new BinanceClient({ apiKey: key, apiSecret: secret });
```

نقل آمن عبر TLS

جميع الاتصالات بين عميل فيكسور وخوادمها تتم عبر **TLS 1.3** (Transport Layer Security). هذا التشفير شاملاً لجميع البيانات أثناء النقل (In-Transit)، مما يمنع التنصت (Eavesdropping) وتعديل البيانات (Tampering) وهجمات إعادة التشغيل (Replay Attacks). تستخدم

• التشفير في حالة السكون (At-Rest)

بيانات الاعتماد المُشفَّرة في قاعدة بيانات Supabase محمية بتشفير على مستوى التخزين (e-Level Encryption) المُفعَّل افتراضياً في Supabase Cloud. مع تشفير التطبيق الإضافي عبر encrypted/crypto تحظى البيانات بطبقتي تشفير مستقلتين: واحدة من Supabase وأخرى من فيكسور.

مكتبات التشفير المعتمدة

- **noble/ed25519@:** تنفيذ خفيف وصارم لخوارزمية Ed25519 للتوقيعات الرقمية وتبادل المفاتيح.
- **tweetnacl:** مكتبة NaCl المُصَغَّرة لتشفير متقاسم (Shared Key Encryption) عبر 20-Poly1305.
- **Web Crypto API:** واجهة التشفير المدمجة في المتصفح للعمليات التي تتم على جانب العميل.

15

الاستجابة للحوادث (Incident Response)

تتبنى فيكسور استراتيجية شاملة للاستجابة للحوادث الأمنية (Incident Response) تهدف إلى تقليل الأخطاء بسرعة، احتواء تأثيرها، واستعادة الخدمة الطبيعية في أقصر وقت ممكن. تتكون الاستراتيجية من أربع مراحل متكاملة: **الكشف** عبر أنظمة المراقبة، **الاحتواء** عبر حدود التحليل عبر السجلات، و**الاسترداد** عبر آليات التراجع.

حدود الخطأ (Error Boundaries)

تُستخدم **Error Boundaries** في React لمنع أخطاء المكونات الفردية من تعطيل التطبيق عند حدوث خطأ غير مُعالَج داخل مكون، يلتقط Error Boundary الخطأ ويُعرض واجهة بديلة للشاشة البيضاء. هذا يضمن أن خطأ في مكون واحد (مثلاً رسم بياني لعملية معينة) لا يؤثر التطبيق. يُطبَّق هذا النمط على مستويات متعددة: حدود عالمية على مستوى التطبيق، وحدود على مستوى الميزات الحرجة.

تنبيهات Sentry الفورية

يرتبط فيكسور بـ **Sentry** عبر DSN خاص يُرسل جميع الأخطاء غير المُعالَجة في الإنتاج للمراقبة المركزية. تُفعَّل التنبيهات الفورية (Real-Time Alerts) التي تُخطر فريق المراقبة عند حدوث أخطاء حرجة. يوفر Sentry أيضاً تجميع الأخطاء (Error Grouping) للأخطاء المتشابهة، وتتبع الإصدار (Release Tracking) لتحديد الإصدار الذي ظهر فيه الخطأ المستخدم (User Context) لفهم تأثير الخطأ.

الاحتواء والاسترداد

Error Boundaries تحتوي الأخطاء على مستوى احتواء. **CircuitBreaker** يحتوي فشل الخدمة الخارجية. **LRU Cache** يوفر بيانات مخزنة كـ **Fallback**. التوزيع عبر **Vercel** يسمح بالتراجع (**Rollback**) إلى إصدار سابق بنجاح واحدة.

الكشف المبكر

Sentry + Mixpanel يكشفان الأخطاء والأنشطة المشبوهة في الوقت الحقيقي. **CircuitBreaker** يكشف تدهور الخدمات الخارجية. **Rate Limiter** يكشف أنماط الهجوم. كل طبقة تُرسل إشارات إنذار تُترابَّك لتشكيل صورة شاملة لحالة النظام.

توحيد الأخطاء الكارثية

يُنقِّد ملف `server.ts` آلية **Catastrophic Error Normalization** التي تُحوِّل جميع الأخطاء إلى `TypeError` موحدة، مما يسهِّل معالجة الأخطاء في بيئة **التطوير**، تُعرض تفاصيل كاملة تتبَّع `Trace` ونوع الخطأ والسياق لمساعدة المطورين في التصحيح. في بيئة **الإنتاج**، تُرجع رسالة ("حدث خطأ داخلي") لحماية المعلومات الحساسة، مع تسجيل التفاصيل الكاملة في `Sentry` لمنع هذا نمط التسريب المعلوماتي (`Information Leakage`) الذي قد يُسهِّل عمل المهاجمين.

➤ التراجع عبر **Vercel**

يستفيد فيكسور من منصة **Vercel** للاستضافة مما يوفر إمكانية التراجع (`Rollback`) إلى أي نشر سابق بنقرة واحدة. في حالة اكتشاف مشكلة حرجية في إصدار جديد، يمكن إعادة توجيه حركة المرور إلى السابق المستقر في ثوانٍ معدودة، مما يُقلل وقت التوقف (`Downtime`) إلى الحد الأدنى.

وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal